

TRACE-ED Simulation Guide

From Environment Setup to Monte Carlo Execution and Analysis

PART I — ENVIRONMENT PREPARATION

1. Install Python

Make sure Python 3.10 or later is installed on the system.

To verify the installation, run:

```
python --version
```

If Python is not yet installed, install it first before continuing with the setup. The TRACE-ED simulation environment was designed to run in a Python-based workflow with external package dependencies for API access, embeddings, numerical computation, and data analysis.

2. Open Command Prompt in the project root

Open Command Prompt and navigate to the main TRACE-ED project folder. For example:

```
cd C:\trace-ed
```

Adjust the path as needed for the local machine. The project root should contain folders such as `00_protocol`, `03_runs`, `04_code`, and the file `config.json`.

3. Create a virtual environment

Inside the project root, create a virtual environment:

```
python -m venv .venv
```

Activate it using:

```
.venv\Scripts\activate
```

When activated successfully, the command line should display the environment marker:

```
(.venv)
```

This step isolates the required dependencies for the TRACE-ED workflow.

4. Install required packages

Install the required Python packages:

```
pip install --upgrade pip
pip install openai
pip install sentence-transformers
pip install numpy
pip install scipy
pip install pandas
```

Optional packages for extended analysis or visualization may also be installed:

```
pip install matplotlib seaborn
```

The main dependencies support experiment execution, embedding-based grounding, and post-run analysis.

5. Set the OpenAI API key

The experiment requires an available OpenAI API key in the environment.

For the current terminal session:

```
set OPENAI_API_KEY=your_api_key_here
```

A persistent environment variable may also be configured at the operating system level if preferred. The execution script initializes the OpenAI client using the `OPENAI_API_KEY` environment variable.

PART II — PROJECT STRUCTURE OVERVIEW

Before running the TRACE-ED simulation, it is important to understand the role of each folder and file in the project structure. Some folders and files must already exist before execution begins, while others are created automatically during the simulation and analysis workflow.

The project root is expected to contain a structure similar to the following:

```
trace-ed/
├── .venv/
├── 00_protocol/
│   ├── pilot_answers_v1.json
│   └── rubric_keywords_v1.json
├── 01_data/
├── 02_prompts/
├── 03_runs/
├── 04_code/
│   ├── run_compare_single_vs_multi.py
│   ├── run_monte_carlo.py
│   └── analyze_monte_carlo.py
├── 05_results/
└── config.json
```

This structure reflects the working project arrangement used in the TRACE-ED study.

1. Project root

The root folder `trace-ed/` is the main working directory of the project. It should contain:

- the configuration file;
- the protocol resources;
- the code files;
- and the folders used for generated outputs.

The simulation and analysis commands are intended to be run from this root directory. This is important because the execution script loads files such as `config.json` and the protocol JSON files using project-relative paths.

2. `.venv/` — virtual environment folder

The `.venv/` folder is generated automatically when the virtual environment is created using:

```
python -m venv .venv
```

This folder does not need to be created manually. It is produced during the environment setup stage described in Part I. It stores the local Python environment and installed packages used to run the simulation. The folder is necessary for local execution, but it is not part of the scientific input data and is generally not needed as a shared research output.

3. 00_protocol/ — protocol input folder

The folder `00_protocol/` must be prepared manually before execution. It is a required input folder and is not generated automatically by the simulation scripts.

This folder contains the core protocol files:

- `pilot_answers_v1.json`
- `rubric_keywords_v1.json`

These files are loaded directly by `run_compare_single_vs_multi.py` during execution. The script explicitly reads them from `00_protocol/`, so the folder and files must already exist before the run starts. If they are missing, the script will fail during the loading step.

In practical terms:

- `pilot_answers_v1.json` provides the question text, rubric structure, and synthetic answers;
- `rubric_keywords_v1.json` provides the rubric-linked keyword definitions used in grounding evaluation.

4. 01_data/ — additional data folder

The folder `01_data/` is part of the working project organization and is typically created manually as a storage location for supporting data files, cleaned datasets, or intermediate analysis-ready materials.

It is not required by the execution script shown in the current workflow, because the main simulation script loads its primary inputs from `00_protocol/` rather than from `01_data/`. Therefore:

- this folder may exist for project organization purposes;
- it is not automatically generated by the environment setup;
- it is not automatically generated by the run script;
- and it does not have to contain files for the core simulation to execute, unless the project is extended.

5. 02_prompts/ — prompt resource folder

The folder `02_prompts/` is also part of the broader project organization and should be created manually if used. It is intended to store prompt templates, prompt variants, or prompt documentation for the single-agent and multi-agent configurations.

In the current script version, the prompt strings are defined directly inside `run_compare_single_vs_multi.py`, which means the simulation can still run even if `02_prompts/` is empty. However, from a reproducibility and documentation perspective, keeping prompt materials in a dedicated folder is recommended.

So, in the present workflow:

- `02_prompts/` is not automatically generated;
- it is not strictly required for code execution if prompts remain hardcoded in the script;
- but it is useful as a manual project documentation folder.

6. `03_runs/` — run output root

The folder `03_runs/` functions as the main output root for experiment execution. It should exist in the project root before running the experiment, as this is the directory specified by `run_root` in `config.json`.

Inside the script:

- the run root is read from `config.json`;
- a timestamped run folder is created automatically;
- and inside that folder the script also creates a `runs/` subfolder automatically.

This means:

- `03_runs/` should be available as the main output directory;
- the timestamped run folder inside it is generated automatically during execution;
- the `runs/` subfolder inside that run folder is also generated automatically.

After a successful run, the structure will look like:

```
03_runs/
  pilot_montecarlo_v1_COMPARE_YYYYMMDD_HHMMSS/
    records.json
    runs/
      single_A1_temp_0.0_rep_1/
      multi_A1_temp_0.0_rep_1/
      ...
```

So the run folder and its contents are not created manually; they are produced automatically by the simulation script.

7. `04_code/` — execution and analysis scripts

The folder `04_code/` must be prepared manually before execution. It is not created by the Python environment installation and not generated automatically by the experiment.

This folder contains the main code files used in the workflow:

- `run_compare_single_vs_multi.py`
- `run_monte_carlo.py`
- `analyze_monte_carlo.py`

At minimum, the current workflow depends directly on:

- `run_compare_single_vs_multi.py` for experiment execution;
- `analyze_monte_carlo.py` for post-run analysis.

Therefore, `04_code/` and its script files must already exist before the simulation starts.

8. `05_results/` — result archive or auxiliary results folder

The folder `05_results/` is a project organization folder that is typically created manually. It is useful for storing exported summaries, tables, figures, or copied final outputs that are separated from raw run folders.

However, in the latest workflow, the main analysis results are written directly into:

```
03_runs/<run_folder>/analysis/
```

rather than into `05_results/`.

So in the current implementation:

- `05_results/` is not automatically generated by the environment installation;
- it is not automatically generated by the simulation script;
- it is not required for the core execution-analysis chain;
- but it may be used manually for curation or archiving of selected result files.

9. `config.json` — main configuration file

The file `config.json` must be created manually before execution. It is not generated automatically by the environment setup or by the simulation scripts.

The execution script opens this file at startup and reads the following types of settings from it:

- experiment name,
- model name,
- question identifier,
- temperature values,
- number of repetitions,
- grounding threshold,
- output root directory.

Because the script depends on it immediately, `config.json` must be present in the project root before the simulation is run.

10. Automatically generated outputs during the workflow

During the execution-analysis process, several folders and files are created automatically.

During simulation

When `run_compare_single_vs_multi.py` is executed, it automatically generates:

- a timestamped run folder inside `03_runs/`;
- a `runs/` subfolder inside that run folder;

- one folder per run condition inside `runs/`;
- `scoring.json` files;
- `explanation.txt` files;
- `full_output.txt` for certain single-agent outputs;
- `records.json` in the run folder root.

During analysis

When `analyze_monte_carlo.py` is executed in the updated workflow, it automatically generates:

- an `analysis/` subfolder inside the selected run folder;
- `analysis_summary_by_architecture.csv`
- `analysis_summary_by_architecture_and_temperature.csv`
- `analysis_icc_by_architecture.csv`
- `analysis_architecture_comparison.csv`
- `analysis_summary_full.json`
- `analysis_report.txt`

So these files and folders do not need to be created manually.

11. Summary: what must exist beforehand and what is auto-generated

Must be prepared manually before running

- `trace-ed/` project root
- `00_protocol/`
- `00_protocol/pilot_answers_v1.json`
- `00_protocol/rubric_keywords_v1.json`
- `04_code/`
- `04_code/run_compare_single_vs_multi.py`
- `04_code/analyze_monte_carlo.py`
- `config.json`

Created during environment setup

- `.venv/`

Typically manual but organizational

- `01_data/`
- `02_prompts/`
- `05_results/`

Generated automatically during simulation

- `03_runs/<timestamped_run_folder>/`
- `03_runs/<timestamped_run_folder>/runs/`
- per-run output folders
- `records.json`
- `scoring.json`

- `explanation.txt`
- `full_output.txt` where applicable

Generated automatically during analysis

- `03_runs/<timestamped_run_folder>/analysis/`
- analysis CSV, JSON, and TXT summary files

12. Recommended interpretation

In summary, the project should be understood as having:

- a manual preparation layer: protocol files, code files, and configuration;
- an environment setup layer: the virtual environment and installed packages;
- an automatic execution layer: run folders and raw output files;
- an automatic analysis layer: the `analysis/` subfolder and derived summaries.

This distinction helps ensure that the TRACE-ED workflow can be reconstructed correctly and that missing components can be diagnosed more easily before execution begins.

PART III — DATA AND RUBRIC FILE LOCATIONS

The core experimental inputs for the TRACE-ED simulation are stored in the `00_protocol/` directory. This folder contains the synthetic short-answer dataset and the rubric-linked keyword definitions used during execution and grounding analysis. The execution script reads these files directly at runtime.

1. Synthetic answers file

The file:

```
00_protocol/pilot_answers_v1.json
```

contains the synthetic short-answer responses used in the experiment. According to the execution script, this file provides:

- the question text,
- the rubric structure, and
- the list of answers evaluated in the Monte Carlo run.

The relevant section of the script loads it as follows:

```
with open("00_protocol/pilot_answers_v1.json", "r", encoding="utf-8") as f:
    DATA = json.load(f)

question_text = DATA["question_text"]
rubric = DATA["rubric"]
answers = DATA["answers"]
```

2. Rubric keyword file

The file:

```
00_protocol/rubric_keywords_v1.json
```

contains rubric-related keyword definitions used in the grounding process. These keywords are matched to the configured `question_id` and support the TRACE-ED grounding logic, including absence-claim handling and semantic traceability evaluation.

The script loads it as:

```
with open("00_protocol/rubric_keywords_v1.json", "r", encoding="utf-8") as f:
    RUBRIC_KEYWORDS = json.load(f)

EXPECTED_KEYWORDS = RUBRIC_KEYWORDS.get(QUESTION_ID, {})
```

3. Practical role of both files

In practical terms:

- `pilot_answers_v1.json` stores the synthetic responses and rubric structure used as the main experimental input;
- `rubric_keywords_v1.json` stores the expected rubric-linked keywords used during grounding analysis.

Both files must remain in the expected location. If either file is missing or moved outside the project structure, the simulation script will fail during loading.

4. Recommendation before execution

Before beginning either a dry run or the full simulation, verify that:

- `00_protocol/` exists in the project root;
- `pilot_answers_v1.json` is present;
- `rubric_keywords_v1.json` is present; and
- the `question_id` in `config.json` corresponds to the protocol resources being used.

PART IV — CONFIGURATION

The TRACE-ED simulation is controlled through a single configuration file located in the project root:

```
trace-ed/config.json
```

This file is a required part of the workflow. It must exist before the simulation starts, and it is read immediately by the execution script. The script loads the experiment settings from this file at startup and uses them to determine:

- which model will be called;
- which question configuration will be used;
- how many repeated runs will be executed;
- which temperature values will be tested;
- which grounding threshold will be applied;
- where the run outputs will be stored;
- and how the experiment folder will be named.

1. Purpose of `config.json`

The purpose of `config.json` is to centralize the core experimental settings so that:

- the simulation can be controlled without editing the main Python script;
- the run configuration can be documented clearly;
- the workflow remains reproducible across repeated runs.

In practice, this file acts as the control panel for the TRACE-ED simulation. Any major change to the experiment design should be reflected first in `config.json` before the script is run.

2. Example configuration

A typical configuration file is structured as follows:

```
{
  "experiment_name": "pilot_montecarlo_v1",
  "model": "gpt-4.1-mini",
  "question_id": "Q-PILOT-001",
  "temperatures": [0.0, 0.2, 0.7],
  "n_repeats": 15,
  "tau_grounding": 0.75,
  "run_root": "03_runs"
}
```

3. Parameters that are required

The following parameters are required for the current script to run correctly.

experiment_name

This parameter is required. It is used to label the run and to construct the timestamped output folder name.

For example, if:

```
"experiment_name": "pilot_montecarlo_v1"
```

then the script may generate a folder such as:

```
03_runs/pilot_montecarlo_v1_COMPARE_20260421_200317/
```

This makes the experiment easier to identify and differentiate from other runs.

model

This parameter is required. It defines which LLM model is used in the simulation.

Example:

```
"model": "gpt-4.1-mini"
```

The execution script reads this value and uses it in the API request for both the single-agent and multi-agent workflows. Changing this parameter changes the underlying model used to generate the scoring and explanation outputs.

question_id

This parameter is required. It identifies which question configuration should be used.

Example:

```
"question_id": "Q-PILOT-001"
```

This value is important because it is used to retrieve the relevant rubric-linked keyword definitions from `rubric_keywords_v1.json`. If the `question_id` does not match an available entry in the protocol files, the grounding logic may fail or produce incomplete results.

temperatures

This parameter is required. It defines the list of temperature settings tested in the simulation.

Example:

```
"temperatures": [0.0, 0.2, 0.7]
```

These values are iterated during execution. In the multi-agent workflow, the scoring step is forced to `temperature = 0.0`, while the explanation step uses the configured temperature values. In the single-agent workflow, the configured temperature values are used directly. This means the contents of the `temperatures` list directly affect the number of run conditions.

n_repeats

This parameter is required. It defines how many times each condition is repeated.

Example:

```
"n_repeats": 15
```

This parameter is one of the main drivers of total run count. It is commonly reduced temporarily during a dry run and restored before the full experiment.

tau_grounding

This parameter is required. It defines the similarity threshold used in the grounding evaluation.

Example:

```
"tau_grounding": 0.75
```

The execution script uses this threshold when deciding whether a claim is considered grounded. Changing this value changes the strictness of grounding classification.

run_root

This parameter is required. It specifies the root output folder where experiment runs will be stored.

Example:

```
"run_root": "03_runs"
```

The execution script uses this value to create the run directory and all subsequent output folders. In the current project structure, this should normally remain 03_runs.

4. Parameters that can be changed safely

The following parameters may be changed, but they should be changed deliberately and documented.

experiment_name

This can be changed safely when you want to distinguish one experiment from another. It is especially useful when:

- testing a revised dataset;
- testing a different prompt strategy;
- testing a different set of temperatures;
- or running a second version of the same study.

Recommended practice: rename the experiment when a meaningful protocol or configuration change is made. This is also consistent with the recommendation not to change temperature settings without renaming the experiment.

model

This can be changed if you intentionally want to compare or migrate to another model. However, changing the model means the run should be treated as a distinct experiment version, because the outputs may differ substantially.

temperatures

This can be changed, but doing so changes the number of experimental conditions and therefore the total number of runs. It may also affect score variability and explanation behavior, since temperature is one of the stochastic controls in the experiment.

n_repeats

This can be changed safely for testing or scaling, but it directly changes the total run count. For example:

- `n_repeats = 2` is suitable for a dry run;
- `n_repeats = 15` is suitable for the full experiment.

tau_grounding

This can be changed if the study design requires a different grounding criterion, but changing it affects the Grounding Rate calculation and should therefore be treated as a meaningful methodological change.

5. Parameters that normally should not be changed casually

Although technically editable, some parameters should not be changed without a clear reason.

question_id

This should remain stable unless you are intentionally switching to another question setup in the protocol files. If changed incorrectly, it may break the link between:

- the answer set,
- the rubric structure,
- and the grounding keywords.

run_root

This should usually remain `03_runs`. Changing it is possible, but it will redirect all output generation to another location and may make the workflow harder to follow.

6. How configuration affects total run count

The total number of runs is determined by:

Number of answers × Number of temperatures × Number of repeats × Number of architectures

In the TRACE-ED pilot experiment:

- number of answers = 10
- number of temperatures = 3
- number of repeats = 15
- number of architectures = 2

So the total is:

$$10 \times 3 \times 15 \times 2 = 900 \text{ runs}$$

For the dry run:

- number of answers = 10
- number of temperatures = 3
- number of repeats = 2
- number of architectures = 2

so the total becomes:

$$10 \times 3 \times 2 \times 2 = 120 \text{ runs}$$

This shows that changing `n_repeats` or the number of temperature values directly scales the overall experiment size.

7. How configuration affects run outputs

The configuration does not only affect the number of runs. It also affects:

- the naming of the output folder through `experiment_name`;
- the location of the output folder through `run_root`;
- the grounding decision threshold through `tau_grounding`;
- the content and stochastic behavior of model outputs through `model` and `temperatures`.

As a result, configuration changes should be treated as part of the documented experiment design, not merely as implementation details.

8. Recommended editing workflow

A recommended sequence for editing `config.json` is:

1. open `config.json` in the project root;
2. confirm that `question_id`, `temperatures`, and `run_root` are correct;
3. set `n_repeats = 2` for a dry run if needed;
4. save the file;
5. run the simulation from the project root;
6. confirm the dry run outputs;
7. restore `n_repeats = 15` for the full run;
8. save the file again before starting the full simulation.

This workflow reduces the risk of launching a large run with an incorrect configuration.

9. Practical interpretation of each field

For convenience, the configuration fields may be interpreted as follows:

- `experiment_name`
labels the run folder.
- `model`
selects the LLM used in scoring and explanation generation.
- `question_id`
links the run to a specific question/rubric keyword setup.
- `temperatures`
defines the stochastic conditions tested.
- `n_repeats`
defines how many times each condition is repeated.
- `tau_grounding`
defines the grounding strictness threshold.
- `run_root`
defines where the experiment outputs will be written.

10. Recommended stability rule

Once a full run is started, the configuration should be treated as frozen. Do not change:

- temperatures,
- prompts,
- question setup,
- or grounding threshold

mid-experiment. If a meaningful change is needed, save it as a new run configuration and preferably use a new `experiment_name`. This keeps the run folders distinguishable and preserves reproducibility.

PART V — DRY RUN (SANITY CHECK)

Before launching the full Monte Carlo simulation, a dry run should be performed as a standard verification step. The purpose of the dry run is to confirm that the project structure, configuration file, protocol inputs, API connection, and output generation are all functioning correctly before committing to the larger execution workload.

In the TRACE-ED workflow, a dry run is a smaller-scale test of the full simulation pipeline. It uses the same core logic as the full run, but with a reduced repetition count so that problems can be detected early and corrected with minimal cost. This is especially important because the full experiment involves repeated API calls, run-folder creation, per-run output generation, and downstream statistical analysis.

1. Purpose of the dry run

The dry run is intended to verify that:

- `config.json` is present and readable;
- the required protocol files are present in `00_protocol/`;
- the script can connect to the model environment successfully;
- run folders are created correctly under `03_runs/`;
- `records.json` is generated;
- run-level files such as `scoring.json` and `explanation.txt` are produced;
- the total number of runs matches the reduced test configuration.

In other words, the dry run checks the end-to-end execution chain on a smaller scale before the full experiment is started.

2. Why the dry run is necessary

The full TRACE-ED simulation uses repeated stochastic execution across multiple answers, temperature values, repetitions, and architectural conditions. In the current pilot design, the full configuration produces 900 runs. Launching the full experiment without first confirming the configuration may lead to avoidable errors such as:

- missing protocol files,
- incorrect `question_id` mapping,
- path errors,
- failure to create output directories,
- or incomplete result generation.

A dry run reduces this risk by testing the same workflow using a smaller number of repetitions.

3. File to edit before the dry run

The dry run is controlled through the configuration file:

```
trace-ed/config.json
```

This is the file where the repetition count should be reduced temporarily. The Python execution script does not require direct editing for this step; instead, the dry run is created by modifying the configuration value in `config.json`.

4. Parameter to change for the dry run

Open `config.json` and locate the line:

```
"n_repeats": 15
```

Temporarily change it to:

```
"n_repeats": 2
```

This reduces the run size while keeping the rest of the experiment structure unchanged. The dry run therefore remains representative of the full workflow, but at a lower execution cost.

A dry-run configuration may look like this:

```
{
  "experiment_name": "pilot_montecarlo_v1",
  "model": "gpt-4.1-mini",
  "question_id": "Q-PILOT-001",
  "temperatures": [0.0, 0.2, 0.7],
  "n_repeats": 2,
  "tau_grounding": 0.75,
  "run_root": "03_runs"
}
```

5. What remains unchanged during the dry run

Only the repetition count should be reduced for the dry run. The following should normally remain unchanged:

- `experiment_name`
- `model`
- `question_id`
- `temperatures`
- `tau_grounding`
- `run_root`

This ensures that the dry run tests the same simulation logic as the intended full experiment, rather than testing a substantially different configuration.

6. Expected number of runs in the dry run

The total number of runs is determined by:

Number of answers × Number of temperatures × Number of repeats × Number of architectures

For the dry run:

- number of answers = 10
- number of temperatures = 3
- number of repeats = 2
- number of architectures = 2

So the expected dry-run total is:

$10 \times 3 \times 2 \times 2 = 120$ runs

This is substantially smaller than the full 900-run simulation and is therefore suitable for early verification.

7. Recommended checks before running

Before executing the dry run, confirm the following:

- the virtual environment is active;
- `config.json` has been saved after editing `n_repeats`;
- `00_protocol/pilot_answers_v1.json` is present;
- `00_protocol/rubric_keywords_v1.json` is present;
- `04_code/run_compare_single_vs_multi.py` is present;
- the `OPENAI_API_KEY` environment variable is available in the active terminal session.

These checks help reduce the risk of failure before execution starts.

8. Where to run the command

The dry run should be launched from the project root, not from inside `04_code/`.

From the root folder, run:

```
python 04_code\run_compare_single_vs_multi.py
```

This is important because the script reads `config.json` and the protocol files using project-relative paths such as `config.json`, `00_protocol/rubric_keywords_v1.json`, and `00_protocol/pilot_answers_v1.json`. Running from the project root ensures those paths resolve correctly.

9. What happens during the dry run

When the command is executed, the script performs the same overall procedure as in the full experiment:

- reads the configuration file;
- loads the rubric keywords;
- loads the synthetic answers and rubric structure;
- creates a timestamped run folder;
- creates a `runs/` subfolder;
- generates outputs for each architecture, answer, temperature, and repetition condition;
- computes the TRACE-ED run-level metrics;

- saves the aggregated `records.json` file;
- prints an ICC summary to the terminal.

This means the dry run is a true small-scale execution of the full simulation pipeline, not a separate simplified script.

10. Expected output location

If the dry run completes successfully, a new folder will appear under:

```
03_runs/
```

with a structure similar to:

```
03_runs/
  pilot_montecarlo_v1_COMPARE_YYYYMMDD_HHMMSS/
    records.json
    runs/
      single_A1_temp_0.0_rep_1/
      multi_A1_temp_0.0_rep_1/
      ...
```

The folder name includes the configured `experiment_name` plus a timestamp generated at runtime. Inside the `runs/` folder, the script creates one subfolder per execution condition.

11. Files expected after a successful dry run

At minimum, the dry run should generate:

- a timestamped run folder inside `03_runs/`;
- `records.json` in the run folder root;
- a `runs/` subfolder;
- `scoring.json` files within run-level folders;
- `explanation.txt` files within run-level folders;
- and, for some single-agent cases, `full_output.txt`.

These outputs confirm that the simulation script is functioning across both the structured scoring and explanation-generation components.

12. Expected terminal output

A successful dry run should also produce terminal output similar to:

```
Multi-agent missing total_score count: 0

=== ICC REPORT (total_score) ===
ICC(1,k) total_score - single: Undefined (zero variance or perfect stability)
ICC(1,k) total_score - multi: 0.99xx

COMPARE run saved to: 03_runs\pilot_montecarlo_v1_COMPARE_...
Total runs: 120
```

The exact ICC value and folder timestamp may vary, but the terminal should confirm:

- the run folder location,
- the total number of runs,
- and the computed ICC reporting behavior.

13. How to interpret dry-run success

The dry run may be considered successful if all of the following are true:

- the script finishes without raising an error;
- the run folder is created in `03_runs/`;
- `records.json` is present;
- the `runs/` folder contains per-condition outputs;
- the terminal reports the expected run count of 120;
- no obvious missing-output problems are observed.

A successful dry run indicates that the configuration, protocol files, and basic execution pipeline are functioning as intended.

14. What to do if the dry run fails

If the dry run fails, do not proceed immediately to the full simulation. Instead, inspect:

- whether `config.json` is present and correctly edited;
- whether the command was run from the project root;
- whether the protocol files exist in `00_protocol/`;
- whether the virtual environment is active;
- whether the API key is available;
- whether the run script can access the required paths.

The dry run is specifically meant to catch these issues before the full 900-run execution is launched.

15. What to do after the dry run succeeds

Once the dry run is confirmed to be successful, reopen `config.json` and restore:

```
"n_repeats": 15
```

Then save the file again. This returns the experiment to the full intended configuration.

After that, proceed to the full Monte Carlo run using the same command:

```
python 04_code\run_compare_single_vs_multi.py
```

16. Recommended stability rule

After a successful dry run, the configuration should be treated as stable. Avoid changing:

- the temperature list,
- the question setup,
- the grounding threshold,
- the prompts,
- or the model

between the dry run and the full run unless you intend to create a new experimental version. If a meaningful change is made, it is recommended to revise the `experiment_name` accordingly so that the run folders remain distinguishable.

17. Summary of the dry-run workflow

The recommended dry-run workflow is:

1. open `config.json`;
2. change `n_repeats` from 15 to 2;
3. save the file;
4. confirm the required protocol and code files are present;
5. run the simulation from the project root;
6. check that the total run count is 120;
7. inspect `records.json` and the `runs/` subfolder;
8. restore `n_repeats` to 15 after success.

This makes the dry run a standard readiness check rather than an optional convenience step.

PART VI — FULL MONTE CARLO RUN

Once the dry run has completed successfully and the configuration has been restored to the intended full setting, the TRACE-ED simulation can proceed to the full Monte Carlo execution. This stage runs the complete experiment under the defined configuration and generates the full output set used for subsequent analysis.

The full run uses the same execution logic as the dry run, but with the complete repetition count and therefore the full number of experimental conditions. In the current TRACE-ED pilot design, this produces the full 900-run dataset described in the study workflow.

1. Transition from dry run to full run

The full run should only be launched after the dry run has succeeded. A successful dry run indicates that:

- the configuration file can be read correctly;
- the protocol files are present and valid;
- the project-relative paths are resolving correctly;
- the API environment is working;
- the output directories can be created successfully;
- and the execution pipeline is generating `records.json` and run-level outputs as expected.

In practical terms, the dry run acts as a readiness check for the full Monte Carlo simulation. Once this check has passed, the same workflow can be executed at full scale.

2. Restore the intended repetition count

Before starting the full run, reopen the configuration file:

```
trace-ed/config.json
```

Locate the dry-run setting:

```
"n_repeats": 2
```

and restore it to:

```
"n_repeats": 15
```

This is the main change required when moving from the dry run to the full experiment. The rest of the configuration should normally remain unchanged unless the experiment is intentionally being redesigned.

A full-run configuration may therefore look like:

```
{  
  "experiment_name": "pilot_montecarlo_v1",  
  "model": "gpt-4.1-mini",  
}
```

```

"question_id": "Q-PILOT-001",
"temperatures": [0.0, 0.2, 0.7],
"n_repeats": 15,
"tau_grounding": 0.75,
"run_root": "03_runs"
}

```

3. What should remain stable before the full run

After a successful dry run, the following parameters should normally remain fixed:

- `experiment_name`
- `model`
- `question_id`
- `temperatures`
- `tau_grounding`
- `run_root`

These settings define the experiment itself. Changing them between the dry run and the full run means the full run is no longer testing the same configuration that was just verified. If a meaningful change is necessary, it is recommended to treat the new setup as a distinct experiment version and rename `experiment_name` accordingly.

4. Full-run scale and expected total run count

The total number of runs is determined by:

Number of answers × Number of temperatures × Number of repeats × Number of architectures

For the full TRACE-ED pilot simulation:

- number of answers = 10
- number of temperatures = 3
- number of repeats = 15
- number of architectures = 2

Therefore, the expected total is:

$10 \times 3 \times 15 \times 2 = 900$ runs

5. Recommended checks before launching the full run

Before running the full simulation, confirm the following:

- the dry run was completed successfully;
- `config.json` has been saved after restoring `n_repeats = 15`;
- the virtual environment is active;
- the required files in `00_protocol/` are present;
- `run_compare_single_vs_multi.py` is present in `04_code/`;
- the API key is available in the current environment;

- sufficient API quota is available for the full run.

Because the full run requires a much larger number of API calls than the dry run, it is useful to confirm quota availability before starting execution.

6. Where to run the full simulation

The full run should be launched from the project root using:

```
python 04_code/run_compare_single_vs_multi.py
```

This should be done from the root directory of the project, not from inside `04_code/`, because the script reads `config.json` and protocol files using project-relative paths. Running from the root ensures those paths resolve correctly.

7. What the full simulation does

When the full Monte Carlo run begins, the script performs the following sequence:

1. reads the configuration file;
2. loads the configured model, temperatures, repeat count, grounding threshold, question identifier, and output root;
3. loads rubric-linked keyword definitions from `rubric_keywords_v1.json`;
4. loads the synthetic answers, question text, and rubric from `pilot_answers_v1.json`;
5. creates a timestamped run folder under `03_runs/`;
6. creates a `runs/` subfolder inside the run directory;
7. iterates through:
 - architecture,
 - answer,
 - temperature,
 - and repetition conditions;
8. generates scoring and explanation outputs;
9. computes TRACE-ED run-level metrics;
10. stores per-run files;
11. saves the aggregated `records.json`;
12. prints ICC and run summary information to the terminal.

This means the full run is the complete realization of the TRACE-ED simulation design, not merely a batch repetition step.

8. Expected runtime behavior

The original guide estimates that the full run may take approximately 20–45 minutes, depending on connection conditions and system performance. During this period:

- do not close the terminal;
- avoid putting the computer into sleep mode;
- avoid editing prompts, configuration, or data files mid-run;
- avoid interrupting the environment unless absolutely necessary.

Because the full run depends on repeated API access and continuous output generation, interruptions may leave incomplete folders or partial results.

9. Expected output location

If the full run completes successfully, a new timestamped run folder should appear inside:

```
03_runs/
```

For example:

```
03_runs/pilot_montecarlo_v1_COMPARE_20260421_200317/
```

Inside this folder, the expected structure is:

```
03_runs/
  pilot_montecarlo_v1_COMPARE_YYYYMMDD_HHMMSS/
    records.json
    runs/
      single_A1_temp_0.0_rep_1/
      multi_A1_temp_0.0_rep_1/
      ...
```

This structure is consistent with the actual behavior of the execution script. The script constructs the run folder name automatically from `experiment_name` plus a timestamp, and then creates the `runs/` subfolder programmatically.

10. Files expected after successful completion

At minimum, a successful full run should produce:

- a timestamped run folder inside `03_runs/`;
- `records.json` in the run folder root;
- a `runs/` subfolder;
- one run-level folder per condition;
- `scoring.json` files in each run folder;
- `explanation.txt` files in each run folder;
- and, for single-agent outputs where applicable, `full_output.txt`.

The `records.json` file is especially important because it serves as the main aggregated dataset used in the recommended analysis step.

11. Expected terminal output after completion

A successful full run should also print terminal output confirming the execution outcome. Based on the execution script, the terminal will report:

- the count of missing multi-agent total scores, if any;
- the ICC report for total score;
- the saved run directory;
- the total number of runs.

A typical output pattern is:

```
Multi-agent missing total_score count: 0

=== ICC REPORT (total_score) ===
ICC(1,k) total_score - single: Undefined (zero variance or perfect stability)
ICC(1,k) total_score - multi: 0.99xx

COMPARE run saved to: 03_runs/pilot_montecarlo_v1_COMPARE_...
Total runs: 900
```

The exact ICC value and timestamp may vary, but the final run count should match the expected total of 900.

12. How to interpret a successful full run

The full Monte Carlo run may be considered successful if all of the following conditions are satisfied:

- the script finishes without raising an exception;
- a new timestamped run folder is created;
- `records.json` is present in the run root;
- the `runs/` subfolder contains the expected run-level outputs;
- the terminal reports `Total runs: 900`;
- no obvious data-loss or missing-score issue is reported.

A successful full run indicates that the experiment has been executed completely and is ready for the post-run analysis stage.

13. What to do if the full run is interrupted or fails

If the run is interrupted or fails before completion, do not assume the outputs are valid for final analysis. Instead:

- inspect whether the run folder is incomplete;
- check whether `records.json` exists and appears complete;
- verify whether all run conditions were generated;
- review the terminal error message;
- correct the issue before restarting or rerunning the experiment.

Because the output folder is timestamped automatically, restarting the run usually produces a new run folder rather than overwriting the earlier attempt. This helps preserve incomplete runs for troubleshooting while keeping the final run distinct.

14. Recommended next step after the full run

Once the full Monte Carlo run has finished successfully, the next required step is to perform the post-run analysis using:

```
python 04_code/analyze_monte_carlo.py
```

This analysis step reads `records.json` and writes the derived statistical summaries into the `analysis/` subfolder inside the selected run directory. In the current TRACE-ED workflow, this is the standard continuation of the execution process.

15. Summary of the full-run workflow

The recommended full-run workflow is:

1. confirm that the dry run succeeded;
2. restore `n_repeats = 15` in `config.json`;
3. save the configuration file;
4. verify protocol files, environment, and quota;
5. run the simulation from the project root;
6. wait until the script completes without interruption;
7. confirm that `records.json` and `runs/` have been generated;
8. verify that the terminal reports `Total runs: 900`;
9. proceed immediately to the analysis stage.

This makes the full Monte Carlo run the direct extension of the dry-run stage, using the same verified pipeline at the intended experiment scale.

PART VII — VERIFY RUN OUTPUT

After the full Monte Carlo simulation has completed, the next step is to verify that the run outputs were generated correctly before proceeding to the analysis stage. This verification step is important because the post-run analysis depends on the completeness and internal consistency of the generated files, especially the aggregated `records.json` file and the run-level outputs stored in the `runs/` subfolder.

In the TRACE-ED workflow, run verification should be treated as a standard checkpoint between execution and analysis. A successful run does not only mean that the script finished; it also means that the expected output structure and core files were produced in a usable form.

1. Purpose of output verification

The purpose of output verification is to confirm that:

- the full simulation completed without structural failure;
- a new timestamped run folder was created in the expected location;
- the aggregated run file `records.json` exists;
- the `runs/` subfolder exists and contains the run-level outputs;
- the total number of generated conditions is consistent with the configured experiment size;
- the outputs are ready for the recommended analysis step.

This step reduces the risk of running downstream analysis on incomplete or partially generated output.

2. Expected output location

The execution script creates its outputs under the folder defined by `run_root` in `config.json`. In the current workflow, this is typically:

```
03_runs/
```

After a successful run, a timestamped experiment folder should be created automatically inside this directory. For example:

```
03_runs/pilot_montecarlo_v1_COMPARE_20260421_200317/
```

The actual folder name will vary depending on the execution date and time, because the script appends a timestamp to the configured `experiment_name`.

3. Expected folder structure

A successful run should produce a structure similar to the following:

```
03_runs/  
  pilot_montecarlo_v1_COMPARE_YYYYMMDD_HHMMSS/  
    records.json  
  runs/
```

```
single_A1_temp_0.0_rep_1/  
multi_A1_temp_0.0_rep_1/  
...
```

This structure reflects the actual behavior of the simulation script:

- the run folder is created automatically;
- `records.json` is written to the run root;
- a `runs/` subfolder is created automatically;
- and one run-level folder is created for each condition.

4. The importance of `records.json`

The file:

```
records.json
```

is the main aggregated output of the full simulation. It is particularly important because the downstream analysis step reads this file directly and uses it as the structured dataset for summarization and architecture comparison.

The execution script stores one record per run and includes fields such as:

- `answer_id`
- `arch`
- `temperature`
- `repeat`
- `total_score`
- `grounding_rate`
- `mean_coherence`
- `contradiction_rate`

If `records.json` is missing, incomplete, or corrupted, the analysis stage should not proceed until the issue has been resolved.

5. The purpose of the `runs/` subfolder

The folder:

```
runs/
```

contains the individual outputs for each execution condition. For each architecture, answer, temperature, and repetition combination, the script creates a dedicated subfolder and stores run-level artifacts there.

These run-level folders typically contain:

- `scoring.json`
- `explanation.txt`
- and, in the single-agent case where applicable, `full_output.txt`.

This structure supports auditability because it preserves the individual scoring and explanation outputs that underlie the aggregated metrics in `records.json`.

6. What should be checked immediately after the run

After the simulation finishes, perform the following checks in the generated run folder:

1. confirm that a new timestamped run folder was created in `03_runs/`;
2. open the run folder and confirm that `records.json` exists;
3. confirm that the `runs/` subfolder exists;
4. inspect several run-level folders to verify that `scoring.json` and `explanation.txt` are present;
5. if applicable, confirm that `full_output.txt` appears in relevant single-agent folders;
6. confirm that the folder structure appears complete and not partially truncated.

These checks are sufficient to determine whether the output folder is structurally ready for analysis.

7. Expected run count and structural interpretation

For the full TRACE-ED pilot simulation, the expected run count is:

$$10 \times 3 \times 15 \times 2 = 900 \text{ runs}$$

This means the output structure should reflect all conditions formed by:

- 10 answers,
- 3 temperatures,
- 15 repetitions,
- and 2 architectures.

The terminal output printed by the execution script should confirm this with:

```
Total runs: 900
```

If the terminal output does not report the expected total, or if the run folder appears incomplete, the run should be reviewed before the analysis stage begins.

8. Expected terminal confirmation

At the end of the run, the terminal should typically display:

- the missing `total_score` count for multi-agent outputs;
- the ICC report for total score;
- the location of the saved run folder;
- the total number of runs.

A typical output pattern is:

```
Multi-agent missing total_score count: 0
```

```
=== ICC REPORT (total_score) ===  
ICC(1,k) total_score - single: Undefined (zero variance or perfect stability)  
ICC(1,k) total_score - multi: 0.99xx  
  
COMPARE run saved to: 03_runs/pilot_montecarlo_v1_COMPARE_...  
Total runs: 900
```

This terminal output should be interpreted together with the generated files. The terminal alone is not sufficient; the existence of the expected output files must also be confirmed.

9. Minimum criteria for a valid run output

The run output may be treated as valid for the next stage if all of the following conditions are satisfied:

- the script completed without a fatal error;
- a new run folder exists in `03_runs/`;
- `records.json` is present in the run root;
- `runs/` is present;
- run-level subfolders contain the expected files;
- the terminal reports the expected total number of runs;
- no obvious file-generation failures are visible.

These criteria are intended to ensure that the analysis step begins from a complete and structurally coherent run output set.

10. Suggested spot-check of run-level contents

Although it is not necessary to inspect every subfolder manually, it is recommended to spot-check a small number of run folders. For example:

- one single-agent folder;
- one multi-agent folder;
- one folder from a lower temperature condition;
- one folder from a higher temperature condition.

In those folders, confirm that:

- `scoring.json` contains structured scoring output;
- `explanation.txt` contains the generated explanation text;
- any optional `full_output.txt` file is readable where present.

This provides an additional confidence check that the simulation did not only create folders, but also populated them meaningfully.

11. What to do if files appear incomplete

If the run folder exists but the output appears incomplete, do not proceed immediately to analysis. Instead:

- inspect whether the script may have been interrupted;
- check whether `records.json` was written fully;
- review the terminal output for warnings or errors;
- verify whether all expected run-level folders were created;
- confirm that the run count in the terminal matches the intended configuration.

Partial or interrupted outputs should be treated cautiously, especially if the total run count is lower than expected.

12. Relationship between verification and analysis

The recommended analysis script depends on the presence of a valid `records.json` file and a coherent run folder structure. Once the output verification step confirms that the simulation folder is complete, the next step is to run:

```
python 04_code/analyze_monte_carlo.py
```

The analysis script will then read the selected `records.json` file and generate the derived summaries in the `analysis/` subfolder inside the run directory.

13. Recommended interpretation of this checkpoint

This verification stage should be understood as the bridge between execution and statistical summarization. It confirms that the raw and aggregated outputs required for analysis have been produced correctly and that the run folder is in a suitable state for post-run processing.

In practice, the sequence is:

1. complete the full Monte Carlo run;
2. verify the existence and completeness of the run folder;
3. confirm the presence of `records.json` and `runs/`;
4. then continue to the analysis stage.

14. Summary of the verification workflow

The recommended verification workflow is:

1. locate the newest timestamped run folder in `03_runs/`;
2. confirm that `records.json` exists;
3. confirm that `runs/` exists;
4. inspect selected run-level folders;
5. verify that the terminal output reported `Total runs: 900`;
6. confirm that the output is structurally complete;
7. proceed to `analyze_monte_carlo.py`.

This makes run-output verification a standard checkpoint rather than an informal post-run glance.

PART VIII — RECOMMENDED ANALYSIS AND OUTPUT GENERATION

After the run outputs have been verified, the next required step is to perform the post-run analysis. In the current TRACE-ED workflow, this stage is no longer treated as optional. It is the standard continuation of the Monte Carlo execution process and is required to transform the aggregated run data into structured summaries suitable for inspection, reporting, and documentation.

The analysis stage uses the aggregated file `records.json` generated during the simulation run and produces a dedicated set of summary files inside an `analysis/` subfolder within the selected run directory. This design keeps the raw run outputs and the derived analysis outputs clearly separated while preserving them within the same experiment folder.

1. Purpose of the analysis stage

The purpose of the analysis stage is to convert the raw aggregated run output into structured summaries that can be reviewed directly. Specifically, the analysis step is used to:

- summarize the main TRACE-ED metrics across architectures;
- summarize the same metrics by architecture and temperature;
- compute ICC(1,k) for total score by architecture;
- compare single-agent and multi-agent outputs using summary statistics and statistical comparison measures where available;
- produce both human-readable and machine-readable outputs for documentation and review.

This means that the analysis stage is the bridge between the execution outputs and the files that are most useful for interpretation.

2. Input required for analysis

The analysis script depends on the presence of a valid run folder containing:

`records.json`

This file is created by the simulation script at the end of the execution stage and serves as the main structured dataset for analysis. The execution script stores one record per run and includes fields such as:

- `answer_id`
- `arch`
- `temperature`
- `repeat`
- `total_score`
- `grounding_rate`
- `mean_coherence`
- `contradiction_rate`

If `records.json` is missing, incomplete, or not readable, the analysis stage should not proceed.

3. Location of the analysis script

The analysis is performed using the script:

```
04_code/analyze_monte_carlo.py
```

This file must already exist in the project's `04_code/` directory before the analysis stage begins. In the updated workflow, the script is responsible for:

- locating the selected run folder;
- reading `records.json`;
- computing summary statistics;
- computing ICC summaries;
- producing overall architecture comparison outputs;
- creating the `analysis/` subfolder if it does not yet exist;
- saving all derived outputs into that subfolder.

4. Where to run the analysis command

The analysis command should be executed from the project root, using:

```
python 04_code/analyze_monte_carlo.py
```

This is the recommended execution pattern because the script resolves the project structure relative to the project root and expects the run folders to be available under `03_runs/`. Running from the project root also keeps the execution pattern consistent with the simulation step.

5. How the analysis script selects the run folder

In the updated workflow, the analysis script can be configured to:

- analyze a specific run folder by name, or
- automatically select the latest run folder under `03_runs/` that contains `records.json`.

This makes it possible to analyze the most recent completed run without manually rewriting all file paths each time, while still allowing a specific run to be targeted when needed.

6. What happens during the analysis stage

When the analysis script is executed, it performs the following sequence:

1. locates the selected run folder inside `03_runs/`;
2. confirms that `records.json` exists;
3. creates an `analysis/` subfolder inside the selected run directory if it does not already exist;
4. loads the records from `records.json`;
5. identifies the available architectures and temperature values;

6. computes overall summary statistics by architecture;
7. computes summary statistics by architecture and temperature;
8. builds the ICC matrix for `total_score` by architecture;
9. computes ICC(1,k) by architecture;
10. computes overall architecture comparison summaries between single-agent and multi-agent conditions;
11. writes the results into CSV, JSON, and TXT output files inside the `analysis/` subfolder.

This means the analysis stage is not only descriptive; it also prepares the structured outputs that are useful for review and for downstream reporting.

7. Expected analysis output location

For a run folder such as:

```
03_runs/pilot_montecarlo_v1_COMPARE_20260421_200317/
```

the analysis outputs will be stored in:

```
03_runs/pilot_montecarlo_v1_COMPARE_20260421_200317/analysis/
```

More generally, the expected pattern is:

```
03_runs/<run_folder_name>/analysis/
```

This structure is recommended because it preserves the logical grouping of:

- the aggregated run output (`records.json`);
- the run-level folders (`runs/`);
- and the derived analysis outputs (`analysis/`).

8. Files generated by the analysis script

The updated analysis workflow generates the following files in the `analysis/` subfolder:

- `analysis_summary_by_architecture.csv`
Provides summary statistics for each metric across all runs within each architecture.
- `analysis_summary_by_architecture_and_temperature.csv`
Provides summary statistics for each metric separately for each architecture and temperature value.
- `analysis_icc_by_architecture.csv`
Provides ICC(1,k) results for total score by architecture, along with the number of answers and repeated runs used in the ICC matrix.
- `analysis_architecture_comparison.csv`
Provides the overall comparison between single-agent and multi-agent conditions, including means, standard deviations, Welch's t values, p-values, and Cohen's d where available.
- `analysis_summary_full.json`
Provides a machine-readable export containing the combined analysis results.

- `analysis_report.txt`
Provides a human-readable summary of the main analysis outputs.

These files should be understood as the main post-run analysis deliverables of the workflow.

9. Metrics covered by the analysis stage

The analysis script summarizes the main metrics recorded during execution, including:

- `total_score`
- `grounding_rate`
- `mean_coherence`
- `contradiction_rate`

These metrics correspond directly to the data fields generated by the execution script and therefore provide a structured summary of the main quantitative outputs of the TRACE-ED run.

10. Relationship between analysis outputs and run outputs

The run outputs and analysis outputs serve different purposes:

- the run outputs in `runs/` preserve the condition-level artifacts such as `scoring.json` and `explanation.txt`;
- `records.json` aggregates the core metrics across all runs;
- the `analysis/` subfolder contains summarized and derived outputs built from `records.json`.

This layered structure is useful because it supports both:

- auditability at the level of individual outputs;
- and interpretability at the level of architecture- and temperature-level summaries.

11. Expected folder structure after analysis

Once the analysis has completed successfully, the run folder should look like this:

```
03_runs/
  pilot_montecarlo_v1_COMPARE_20260421_200317/
    records.json
  runs/
    single_A1_temp_0.0_rep_1/
    multi_A1_temp_0.0_rep_1/
    ...
  analysis/
    analysis_summary_by_architecture.csv
    analysis_summary_by_architecture_and_temperature.csv
    analysis_icc_by_architecture.csv
    analysis_architecture_comparison.csv
    analysis_summary_full.json
    analysis_report.txt
```

This structure clearly separates:

- aggregated execution data,
- raw run-level artifacts,
- and derived post-run summaries.

12. What should be checked after analysis finishes

After the analysis command completes, verify the following:

- the `analysis/` subfolder exists inside the selected run folder;
- all expected output files are present;
- `analysis_report.txt` can be opened and read;
- the CSV files contain data rows rather than only headers;
- the ICC file is present and readable;
- the JSON file is readable and structurally complete.

These checks confirm that the analysis completed successfully and that the outputs are usable for review or reporting.

13. Minimum criteria for a successful analysis stage

The analysis stage may be considered successful if:

- the script runs without error;
- the `analysis/` subfolder is created;
- all expected output files are written;
- the generated summaries are readable;
- and the output appears consistent with the preceding run structure.

A successful analysis stage indicates that the Monte Carlo outputs have been transformed into a structured summary set suitable for interpretation.

14. Why the analysis stage is recommended rather than optional

In the original guide, analysis was described as optional. In the updated workflow, however, it should be treated as recommended and standard because:

- the raw run outputs alone are not the most convenient basis for interpretation;
- the analysis files provide structured summaries that are easier to inspect;
- the workflow now includes a clean `analysis/` subfolder specifically designed to hold these derived outputs;
- and these summaries are more suitable for review, documentation, and editorial support than a raw `records.json` file alone.

For this reason, the analysis step is now considered a normal and expected continuation of the TRACE-ED simulation workflow.

15. Recommended workflow order

The recommended post-run sequence is:

1. complete the full simulation;
2. verify that the run outputs are present and structurally complete;
3. confirm that `records.json` exists;
4. run `python 04_code/analyze_monte_carlo.py`;
5. open the `analysis/` subfolder;
6. inspect `analysis_report.txt` and the CSV outputs;
7. use the generated summaries for documentation, review, or further reporting.

This makes the analysis stage an integrated part of the TRACE-ED pipeline rather than a separate optional add-on.

16. Summary of the role of the analysis stage

In practical terms, the analysis stage serves three main purposes:

- it converts the raw aggregated run data into structured summaries;
- it organizes those summaries into a dedicated `analysis/` folder;
- it produces a set of outputs that are easier to review and preserve than terminal output alone.

As a result, this stage should be considered the standard endpoint of the computational workflow after simulation and verification have been completed.

PART IX — SAFE PRACTICE RECOMMENDATIONS

This part provides practical recommendations to help keep the TRACE-ED workflow stable, reproducible, and easier to troubleshoot during execution and analysis.

1. Purpose of safe practice recommendations

These recommendations are intended to reduce avoidable errors during the simulation workflow, especially when running a large number of repeated API-based executions.

2. Recommended practices

The following practices are recommended:

- keep a backup copy of `config.json` before making major changes;
- freeze the dataset and rubric files before starting a large run;
- record the model name and key configuration settings used in the run;
- run a dry run before launching the full simulation;
- verify the run outputs before proceeding to analysis;
- keep each experiment in its own timestamped run folder.

3. Practices to avoid

The following practices should be avoided:

- changing prompts during an active run;
- changing `question_id`, `temperatures`, or `tau_grounding` mid-experiment;
- mixing different datasets inside the same experiment workflow without renaming the experiment;
- interrupting the run unless necessary;
- analyzing incomplete or partially generated run outputs.

4. Why these practices matter

The TRACE-ED workflow depends on repeated stochastic execution under a controlled protocol. If key settings or inputs are changed during execution, the resulting outputs may no longer belong to one consistent experiment. This makes interpretation, comparison, and reproducibility more difficult.

5. Recommended interpretation

These recommendations should be treated as workflow discipline rather than optional convenience. They help preserve consistency between:

- the protocol layer;
- the configuration layer;
- the execution layer;
- and the analysis layer.

6. Summary

In practical terms, safe practice recommendations help ensure that the simulation produces a clean, interpretable, and reproducible output set suitable for analysis and documentation.

PART X — API QUOTA MANAGEMENT

The full TRACE-ED simulation requires repeated API calls across all configured answers, temperature settings, repetitions, and architectures. For this reason, API quota should be checked before launching the full run.

1. Purpose of quota planning

Quota planning helps ensure that the simulation can complete without interruption due to usage limits or insufficient remaining balance.

2. Approximate call volume

For the full experiment:

- single-agent architecture uses 1 call per run;
- multi-agent architecture uses 2 calls per run.

With:

- 10 answers,
- 3 temperatures,
- 15 repeats,
- 2 architectures,

the approximate call count is:

- single-agent: $10 \times 3 \times 15 = 450$ calls
- multi-agent: $10 \times 3 \times 15 \times 2 = 900$ calls
- total ≈ 1350 API calls

3. Approximate token usage

A rough estimate is:

- around 800–1200 tokens per call, depending on prompt and output length.

Because total usage may vary by answer length and generated explanation length, the actual token count should be monitored during or after execution.

4. Recommended practice before the full run

Before starting the full 900-run simulation, confirm that:

- the API key is active;
- quota or billing is available;
- the account can support the expected number of calls;
- the dry run has already completed successfully.

5. Practical interpretation

If quota is too limited, the simulation may stop mid-run and produce an incomplete run folder. For that reason, quota checking should be treated as a standard preparation step before the full Monte Carlo execution.

6. Summary

In practical terms, API quota management is not part of the statistical method itself, but it is necessary to ensure that the computational workflow can complete successfully and produce a valid full run ready for analysis.

PART XI — REPRODUCIBILITY CHECKLIST

Before finalizing results or using the outputs for reporting, the TRACE-ED workflow should be checked for reproducibility and internal consistency.

1. Purpose of the reproducibility checklist

The purpose of this checklist is to confirm that the simulation and analysis outputs are complete, interpretable, and consistent with the intended experimental design.

2. Core checks after execution

After the simulation run, verify that:

- `records.json` exists in the run folder;
- the `runs/` subfolder exists;
- the expected run count was produced;
- no obvious run-level folders are missing;
- no critical output files are absent from inspected run folders.

3. Core checks after analysis

After running `analyze_monte_carlo.py`, verify that:

- the `analysis/` subfolder exists;
- all expected analysis files are present;
- `analysis_report.txt` is readable;
- the CSV files contain data rows;
- the JSON summary file is readable and complete.

4. Metric-level checks

The workflow should also confirm that:

- `total_score` values are present where expected;
- ICC reporting is available for the architecture summaries;
- summary statistics are internally consistent with the run outputs;
- the analysis reflects the intended architecture and temperature structure.

5. Structural consistency checks

The final run folder should preserve a clear structure containing:

- `records.json`
- `runs/`
- `analysis/`

This structure helps preserve both raw execution outputs and derived summaries in one coherent experiment folder.

6. Recommended interpretation

A run may be treated as reproducible and ready for reporting if:

- the simulation completed successfully;
- the expected outputs were generated;
- the analysis stage completed successfully;
- and the resulting files are complete and readable.

7. Summary

In practical terms, the reproducibility checklist is the final quality-control step before the TRACE-ED outputs are used for interpretation, documentation, or editorial review.

PART XII — FINAL OUTCOME

After completing the full TRACE-ED workflow, the project should yield a complete experiment folder containing both execution outputs and derived analysis outputs.

1. Expected final components

At the end of the workflow, the experiment should include:

- a timestamped run folder inside `03_runs/`;
- `records.json` as the main aggregated run dataset;
- a `runs/` subfolder containing run-level artifacts;
- an `analysis/` subfolder containing derived summary outputs.

2. Main execution outputs

The execution stage should produce:

- `records.json`;
- `scoring.json` files;
- `explanation.txt` files;
- and, where applicable, `full_output.txt` files.

These files preserve the raw outputs of the simulation process.

3. Main analysis outputs

The analysis stage should produce:

- `analysis_summary_by_architecture.csv`
- `analysis_summary_by_architecture_and_temperature.csv`
- `analysis_icc_by_architecture.csv`
- `analysis_architecture_comparison.csv`
- `analysis_summary_full.json`
- `analysis_report.txt`

These files preserve the derived summaries of the simulation process.

4. Final folder interpretation

In practical terms, the final experiment folder should preserve three layers of information:

- aggregated run data in `records.json`;
- raw condition-level outputs in `runs/`;
- summarized post-run outputs in `analysis/`.

This structure supports both auditability and interpretability.

5. Final workflow interpretation

The completed TRACE-ED workflow therefore provides a reproducible chain from:

- protocol inputs,
- to configuration,
- to simulation execution,
- to aggregated outputs,
- to structured analysis summaries.

6. Summary

At the end of the process, the TRACE-ED project should have produced a clean, organized, and reproducible experiment record that is ready for inspection, reporting, supplementary documentation, or editorial review.